UAV Software Engineering: ROS 2 Architecture

Advanced Object-Oriented C++ for ROS 2

1 The Fallacy of Global State in Avionics

In embedded systems, relying on global variables (as seen in basic multi-threading examples) is an anti-pattern. In a multi-rotor UAV, you might need to instantiate four identical motor controller nodes. If state is global, these nodes will overwrite each other's memory (ending in Data Races).

Object-Oriented Programming (OOP) provides **Encapsulation**. By moving publishers, subscribers, queues, and mutexes into the `private` scope of a C++ class, each node maintains its own isolated memory footprint. The class constructor safely initializes the DDS entities, and the destructor (following the RAII paradigm) cleanly destroys network endpoints if the node crashes or is shut down.

2 Inheritance and Shared Ownership (`rclcpp::Node`)

In ROS 2, every application node strictly inherits from the base class `rclcpp::Node`. This inheritance provides the derived class with an integrated middleware interface (logging, parameters, time, and network topology).

Crucially, ROS 2 manages the lifecycle of these objects using smart pointers (covered in deep-dive_smartptr). A ROS 2 node is rarely instantiated on the stack. Instead, it is instantiated on the heap using `std::make_shared<MyNode>()`. This is mandatory because the ROS 2 **Executor** (the engine that spins the node to check for incoming network traffic) requires shared ownership of the node to ensure it is not destroyed while a network callback is still executing.

3 Hardware Abstraction and the Virtual Table (VTable) Overhead

OOP allows for Hardware Abstraction Layers (HAL) through **Polymorphism**. A flight controller might define an abstract base class `BaseGPS` with a pure virtual function `virtual void read_coordinates() = 0;`. Derived classes (e.g., `UbloxGPS` or `SeptentrioGPS`) override this function. The core control loop only interacts with `BaseGPS` pointers, making the drone hardware-agnostic.

However, in hard real-time C++, polymorphism introduces overhead. When a `virtual` function is called, the compiler cannot resolve the address at compile-time (though the Dynamic Dispatcher). Instead, it looks up the function address in a hidden array of pointers called the **Virtual Table (vtable)** via a Virtual Pointer (vptr) injected into the object's memory. In ultra-low-latency loops (e.g., 4kHz IMU reads), navigating the vtable can cause CPU cache misses. Modern UAV software mitigates this by using the `final` keyword to explicitly terminate inheritance trees, allowing the compiler to devirtualize calls.

4 Event-Driven Execution: `std::bind` vs Lambdas

ROS 2 is event-driven: the middleware triggers a callback when a DDS packet arrives. Pointing the middleware to a class member function is complex because member functions require a hidden `this` pointer to identify the specific object instance.

Modern C++ (C++14/17) strongly prefers **Lambda Expressions** capturing `[this]`. Lambdas are generally superior in robotics because:

1. They are easier for the compiler to inline, reducing function call overhead.
2. They provide strict compile-time type checking, whereas `std::bind` errors often manifest as cryptic template errors.

3. They allow for inline data preprocessing (e.g., converting GPS coordinates from degrees to radians) before passing the data to the main class logic.